HADOOP EXPLORER PLATFORM • TECHNICAL SPECIFICATION




# Hadoop gRPC Replicator

## Системная архитектура, сетевые протоколы, шейпинг и Disaster Recovery

Полный инженерный регламент для системных архитекторов, DevOps-инженеров и SRE администраторов кластеров Big Data

**СТЕК ТЕХНОЛОГИЙ**
Java 21 LTS • Spring Boot 3.3.4 • Netty 4.1 gRPC

**ЦЕЛЕВЫЕ ПЛАТФОРМЫ**
Apache Hadoop 3.x • Hive Metastore • Kerberos SASL

**СТАТУС ДОКУМЕНТА**
Production Specification • Ревизия 1.4

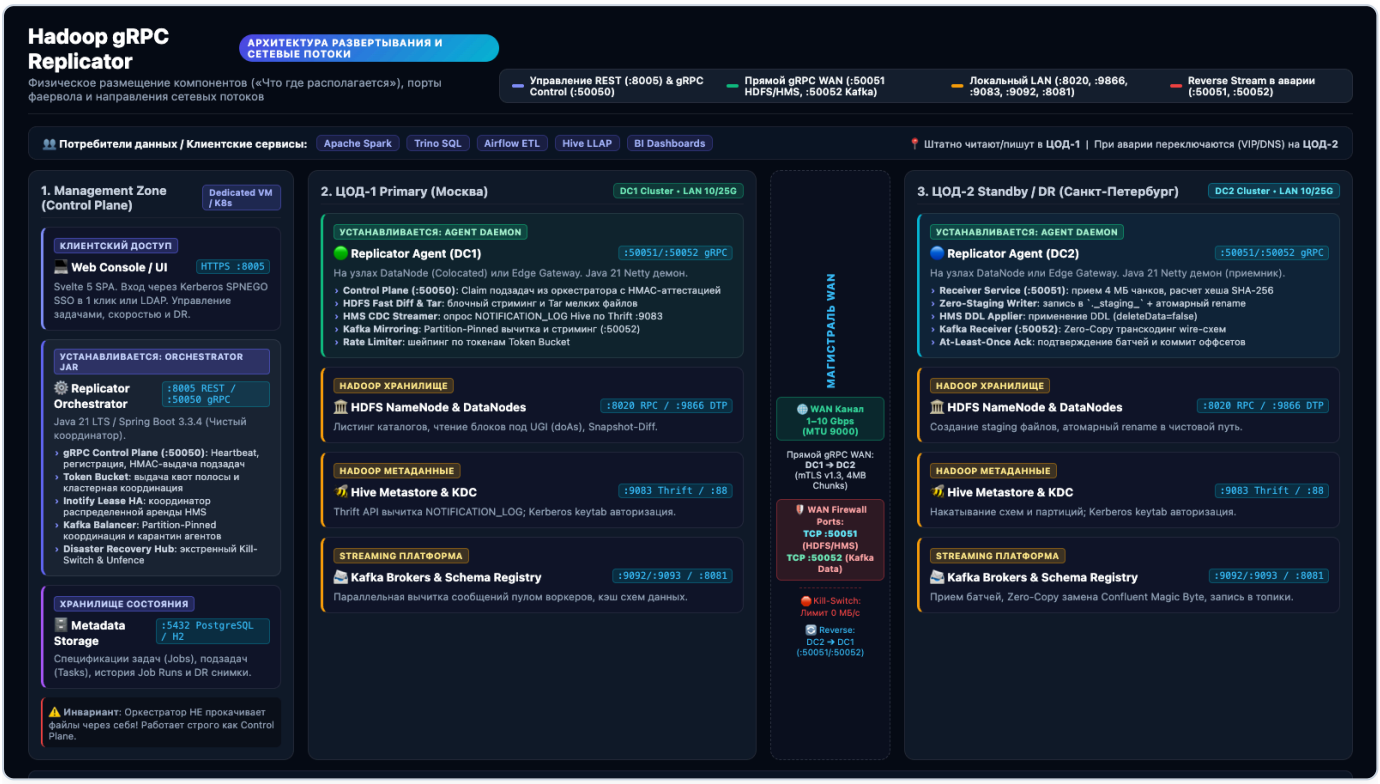
**ДАТА ПУБЛИКАЦИИ**
Октябрь 2026 • Инженерная команда платформы

# Содержание инженерных разделов


1. Слайд 1: Генеральная схема развертывания: Что где располагается (Топология и сетевые потоки)
2. Слайд 2: Спецификация компонентов платформы, роли хостов и инварианты
3. Слайд 3: Модель развертывания, параметры CLI и конфигурация процессов
4. Слайд 4: Сетевая матрица портов, правила фаервола и L4/L7 протоколы
5. Слайд 5: Протокол HDFS Data Plane: Wire-формат, чанкирование и Zero-Staging
6. Слайд 6: Алгоритм сетевого шейпинга: Иерархический Token Bucket
7. Слайд 7: Протокол Hive Metastore CDC, распределенный лизинг и Non-ACID Gate
8. Слайд 8: Потоковая репликация Apache Kafka (Kafka Streaming Data Plane & Mirroring)
9. Слайд 9: Безопасность, Kerberos Impersonation (doAs) и Apache Ranger Audit
10. Слайд 10: SRE Runbook: Авария основного ЦОД и сетевое ограждение (Fencing Kill-Switch)
11. Слайд 11: SRE Runbook: Восстановление, безопасный Unfence и Reverse Replication
12. Слайд 12: Управление платформой в Web UI: Топология, Задачи, HMS, Kafka и DR Hub
13. Слайд 13: Наблюдаемость (Observability): Метрики Prometheus, Healthcheck и Алерты
14. Слайд 14: Сайзинг оборудования, тюнинг ОС Linux, JVM и траблшутинг

СЕКЦИЯ 1 / 13 • ТЕХНИЧЕСКИЙ РЕГЛАМЕНТ

# Слайд 1: Генеральная схема развертывания: Что где располагается (Топология и сетевые потоки)




*Генеральная архитектура: порты Control Plane (TCP :8005 / :50050), межЦОДного WAN Data Plane (TCP :50051 / :50052) и кластерного LAN*

## Разбор физических контуров и направлений трафика

1. **Контур потребителей данных (Clients / Workloads)**:
   - Аналитические движки (Apache Spark, Trino SQL, Airflow ETL, BI Dashboards, Hive LLAP).
   - **Штатный режим**: подключаются и работают исключительно с ЦОД-1 Primary (Москва).
   - **Аварийный режим (DR)**: при объявлении аварии переключаются на ЦОД-2 Standby (Санкт-Петербург) через смену DNS/VIP или переключение конфигураций JDBC/HDFS.

2. **Контур управления (Management Zone / Control Plane)**:
   - Развертывается на выделенной виртуальной машине (VM) или в Kubernetes Pod в управляющем сегменте сети (Management VLAN).
   - **Устанавливается**: `hadoop-grpc-replicator-orchestrator.jar` (Spring Boot 3.3.4, Java 21, `TCP :8005` и `TCP :50050`).
   - Предоставляет REST API и Web-консоль управления на порту `:8005` (Kerberos SPNEGO SSO, LDAP, RBAC), а также высокоскоростной gRPC Control Plane на порту `:50050` (`orchestrator_control_plane.proto`).
   - Хранилище состояния: PostgreSQL 14+ (`TCP :5432` / Patroni HA) с персистентным реестром агентов `agent_registry` (`AgentEntity`), таблицей лимитов `bandwidth_limits` и блокировками `cluster_locks` с монотонными fencing tokens.
   - **Stateless Active-Active кластер**: несколько реплик оркестратора работают совместно без локального in-memory состояния; квоты Token Bucket автоматически распределяются координатором `OrchestratorClusterCoordinator`.
   - Общается с агентами обоих ЦОДов по gRPC Control Plane (`:50050`) и REST API (`:8005`) (регистрация агентов, heartbeat каждые 3–5 с, криптографическая выдача подзадач с HMAC-токенами, распределенный арбитраж Inotify Lease HA).
   - ⚠️ **Архитектурный инвариант**: Оркестратор **НЕ прокачивает байты файлов через себя**, работая исключительно как координатор метаданных.
3. **Контур ЦОД-1 Primary (Москва)**:
   - **Устанавливается**: демоны `hadoop-grpc-replicator-agent.jar` (HDFS/HMS) и `kafka-replicator-agent` (Kafka Streaming Data Plane).
   - Варианты размещения: Colocated на узлах Hadoop DataNode и брокерах Kafka либо на выделенных серверах Edge Gateway.
   - Локально в LAN ЦОД-1 (10/25 Gbps) взаимодействуют со штатными сервисами:
     - **NameNode** (`TCP :8020/:9000`) — получение структуры директорий, списков файлов и снапшотов (`SnapshotDiffReport`);
     - **DataNodes** (`TCP :9866`) — прямое вычитывание блоков HDFS по протоколу SASL Data Transfer Protocol от имени пользователя (`doAs`);
     - **Hive Metastore** (`TCP :9083 Thrift`) — периодический опрос таблицы `NOTIFICATION_LOG` для вычитки DDL/DML событий;
     - **Kafka Brokers** (`TCP :9092/:9093`) — параллельная вычитка сообщений пулом `KafkaBatchConsumerPool` с поддержкой Kerberos SASL;
     - **Schema Registry** (`TCP :8081`) — получение схем данных Avro/Protobuf/JSON;
     - **Kerberos KDC** (`TCP/UDP :88`) — билетная аутентификация по ключевому файлу (`keytab`).
   - **Безопасность и песочница**: локальные файловые пути изолированы в Chroot-песочнице `localSandboxDir` (`REPLICATOR_LOCAL_SANDBOX`) с блокировкой системных каталогов ОС (`FORBIDDEN_OS_PREFIXES`).
4. **МежЦОДная магистраль WAN (Москва ➔ Санкт-Петербург)**:
   - **Единственные порты в межсетевом экране**: **`TCP :50051` (HDFS/HMS gRPC)** и **`TCP :50052` (Kafka Data Plane gRPC, mTLS v1.3, MTU 9000)**.
   - Трафик данных идет **напрямую peer-to-peer от Agent DC1 к Agent DC2**, минуя оркестратор и внутренние кластерные порты.
   - Скорость канала жестко регулируется сетевым шейпером Token Bucket с адаптивным масштабированием лизинга (Adaptive Token Lease Scaling) под RTT и WAN throughput.
   - При аварии активен сетевой Kill-Switch: сброс полосы в 0 МБ/с и монотонный Fencing Token для предотвращения эффекта Split-Brain.

5. **Контур ЦОД-2 Standby / DR (Санкт-Петербург)**:
   - **Устанавливается**: демоны `hadoop-grpc-replicator-agent.jar` (приемник HDFS/HMS на `:50051`) и `kafka-replicator-agent` (приемник Kafka на `:50052`).
   - Слушают входящие порты `TCP :50051` и `TCP :50052`. Серверы Netty переведены на виртуальные потоки Java 21 (`Executors.newVirtualThreadPerTaskExecutor()`).
   - Валидируют криптографическую подпись HMAC-SHA256 задачи (`TaskAttestationValidator`) перед созданием целевых файлов.
   - Принимают чанки HDFS данных по 4 МБ, рассчитывают SHA-256 хеш на лету и пишут во временные файлы `._staging_<jobId>` локальной HDFS (`TCP :9866`).
   - При успешной проверке контрольной суммы вызывают атомарный `fs.rename()` в целевой путь через NameNode DC2 (`TCP :8020`).
   - Мелкие файлы распаковываются из потокового TAR-стрима в выделенном пуле виртуальных потоков `tarUnpackExecutor` с двухфазным коммитом и кэшированием каталогов NameNode `directoryCache` (экономия до 90% RPC).
   - Принимают Hive DDL-пакеты, переписывают URI путей NameService и накатывают схему в локальный Hive Metastore DC2 (`TCP :9083`) с флагом `deleteData=false`.
   - Принимают батчи Kafka по порту `TCP :50052`, транскодируют заголовки схем (`ZeroCopyWireTranscoder`), регистрируют схемы в целевом Schema Registry (`:8081`) и асинхронно записывают сообщения в локальную Kafka (`TCP :9092/:9093`) через `KafkaBatchProducer`.

# Слайд 2: Спецификация компонентов платформы, роли хостов и инварианты

## Назначение и стек технологий

`Hadoop gRPC Replicator` — специализированный сервис межкластерной репликации файлов HDFS, метаданных Hive Metastore (HMS CDC) и очередей Apache Kafka между изолированными дата-центрами (On-Premises, Private Cloud, Hybrid Cloud) с аппаратным контролем полосы WAN и защитой от Split-Brain.

| ПАРАМЕТР | CONTROL PLANE (ОРКЕСТРАТОР) | DATA PLANE (АГЕНТЫ РЕПЛИКАЦИИ) |
|---|---|---|
| **Бинарный пакет** | `hadoop-grpc-replicator-orchestrator.jar` | `hadoop-grpc-replicator-agent.jar` |
| **Среда исполнения** | Java 21 LTS (Spring Boot 3.3.4) | Java 21 LTS (Netty 4.1 на Virtual Threads, tarUnpackExecutor) |
| **Сетевые интерфейсы** | `TCP :8005` (REST, SSE, Web UI), `TCP :50050` (gRPC Control Plane) | `TCP :50051` (gRPC HDFS/HMS), `TCP :50052` (Kafka Data Plane) |
| **База метаданных** | PostgreSQL 14+ ( `agent_registry` , `bandwidth_limits` , `cluster_locks` ) | Отсутствует (Stateless воркеры) |
| **Аутентификация** | Kerberos SPNEGO SSO, LDAP Bind, JWT HttpOnly, Agent Secret | Kerberos Keytab ( `hdfs.headless.keytab` ), mTLS x509, HMAC Attestation |
| **Целевые системы** | Браузеры операторов, Prometheus scraper | Hadoop 3.x (RPC :8020/:9000, DTP :9866), Hive Metastore (:9083), Kafka |

## Базовые архитектурные факты и инварианты

1. **Строгое разделение Control Plane и Data Plane**:
   - Оркестратор отвечает исключительно за координацию: планирование задач, учет воркеров, распределение квот полосы Token Bucket и координацию распределенной аренды HMS Lease.
   - **Байты файлов и DDL-пакеты через Orchestrator НЕ прокачиваются**.
2. **Прямой Peer-to-Peer обмен между агентами по WAN**:
   - Передача данных выполняется напрямую между демонами `replicator-agent-dc1` и `replicator-agent-dc2` по порту `TCP :50051` .
   - В фаерволе между ЦОД открывается ровно один порт. Кластерные порты Hadoop (NameNode, DataNodes) наружу не выставляются.
3. **Stateless-архитектура воркеров и Active-Active Orchestration**:
   - Агенты не сохраняют локальное состояние задач на диск. Реестр агентов хранится в БД ( `agent_registry` ), позволяя запускать оркестраторы в пуле Active-Active с динамическим распределением токенов через `OrchestratorClusterCoordinator` .
4. **Zero-Staging транзакционность и двухфазный коммит**:
   - Файлы на стороне приемника никогда не перезаписываются напрямую. Запись ведется в `._staging_*` , и только после верификации SHA-256 атомарно переименовывается в NameNode. Для TAR-бандлов мелких файлов двухфазный коммит защищает от сбоев и согласуется с `HdfsStagingReaper` .
5. **Криптографическая аттестация задач (HMAC-SHA256)**:
   - Каждая подзадача подписывается сервисом `TaskAttestationService` при выдаче воркеру. Приемник валидирует HMAC перед записью в HDFS через `TaskAttestationValidator` , исключая компрометацию путей и doAs-принципалов.
6. **Chroot Sandbox и защита ОС**:
   - Локальные файловые пути проверяются относительно `localSandboxDir` ( `REPLICATOR_LOCAL_SANDBOX` ) с блокировкой Symlink Traversal, Null Bytes и системных путей Linux/macOS.

# Слайд 3: Модель развертывания, параметры CLI и конфигурация процессов

## Параметры запуска процесса Replicator Orchestrator

```
# Запуск Orchestrator как systemd-сервиса или в Docker-контейнере
java -Xms4g -Xmx8g -XX:+UseG1GC \
  -Dserver.port=8005 \
  -Dspring.datasource.url=jdbc:postgresql://postgres.infra:5432/replicator_db \
  -Dspring.datasource.username=replicator_app \
  -Dspring.datasource.password=${DB_PASSWORD} \
  -jar replicator-orchestrator.jar
```

## Параметры запуска процесса Replicator Agent (CLI Options)

Точка входа: `org.apache.hadoop.explorer.replicator.agent.ReplicatorAgentMain`.

| ПАРАМЕТР CLI | ПЕРЕМЕННАЯ ОКРУЖЕНИЯ | ПО УМОЛЧАНИЮ | ОПИСАНИЕ И НАЗНАЧЕНИЕ |
|---|---|---|---|
| `-id`, `--agent-id` | `AGENT_ID` | `hostname` | Уникальный строковый идентификатор агента в реестре |
| `-c`, `--cluster-id` | `CLUSTER_ID` | `default` | ID обслуживаемого HDFS-кластера (`dc1` или `dc2`) |
| `-m`, `--mode` | `AGENT_MODE` | `all` | Режим работы демона: `all` (sender+receiver), `sender`, `receiver` |
| `-o`, `--orchestrator` | `ORCHESTRATOR_URL` | `http://localhost:8005` | URL управляющего оркестратора для регистрации и heartbeat |
| `-p`, `--port` | `AGENT_RECEIVER_PORT` | `50051` | Порт локального gRPC Netty-сервера для приема входящего WAN трафика |
| `-b`, `--bandwidth` | `AGENT_MAX_BANDWIDTH` | `0` (unlimited) | Локальный лимит пропускной способности агента в МБ/с |
| `-s`, `--staging-dir` | `AGENT_STAGING_DIR` | `/tmp/staging` | Локальный каталог временных метаданных staging |
| `-z`, `--compression` | `WIRE_COMPRESSION` | `zstd` | Алгоритм сжатия потока в канале WAN: `zstd`, `lz4`, `none` |
| `-zl`, `--compression-level` | `ZSTD_LEVEL` | `3` | Уровень сжатия Zstandard (от 1 до 22; оптимум CPU/сеть — 3) |
| — | `REPLICATOR_AGENT_SECRET` | — | Общий секрет взаимной аутентификации агента и оркестратора (токен x-agent-secret) |
| — | `REPLICATOR_LOCAL_SANDBOX` | `/tmp/sandbox` | Корневой каталог Chroot-песочницы локальной ФС с защитой системных путей ОС (SEC-10) |
| — | `REPLICATOR_ENFORCE_TASK_ATTESTATION` | `true` | Обязательная криптографическая проверка HMAC токена задачи перед записью в HDFS |

## Сравнение моделей размещения агентов (Colocated vs Dedicated Gateway)

| ХАРАКТЕРИСТИКА | МОДЕЛЬ COLOCATED (НА УЗЛАХ DATANODE) | МОДЕЛЬ DEDICATED EDGE GATEWAY |
|---|---|---|
| **Физическое размещение** | Устанавливается на физические DataNode кластера | Выделенные 2–4 сервера в стойке с 25G/40G NIC |
| **Локальность данных** | **Short-Circuit Local Read** (чтение блоков напрямую с дисков узла) | Дистанционное чтение блоков по LAN ЦОД (DTP RPC) |
| **Нагрузка на LAN ЦОД** | **0 байт дополнительного трафика** в сети ЦОД | Двойная прокачка (чтение из DataNode + отправка в WAN) |
| **Влияние на Hadoop** | Потребляет 2-4 vCPU и 4-8 GB RAM на узле DataNode | Нулевое влияние на вычислительные ресурсы узлов Hadoop |
| **Рекомендация** | Идеально для кластеров с дисковым I/O запасом | Идеально при жестких политиках изоляции серверов Hadoop |

СЕКЦИЯ 4 / 13 • ТЕХНИЧЕСКИЙ РЕГЛАМЕНТ

# Слайд 4: Сетевая матрица портов, правила фаервола и L4/L7 протоколы

## Сводная сетевая матрица для NetOps и Информационной Безопасности

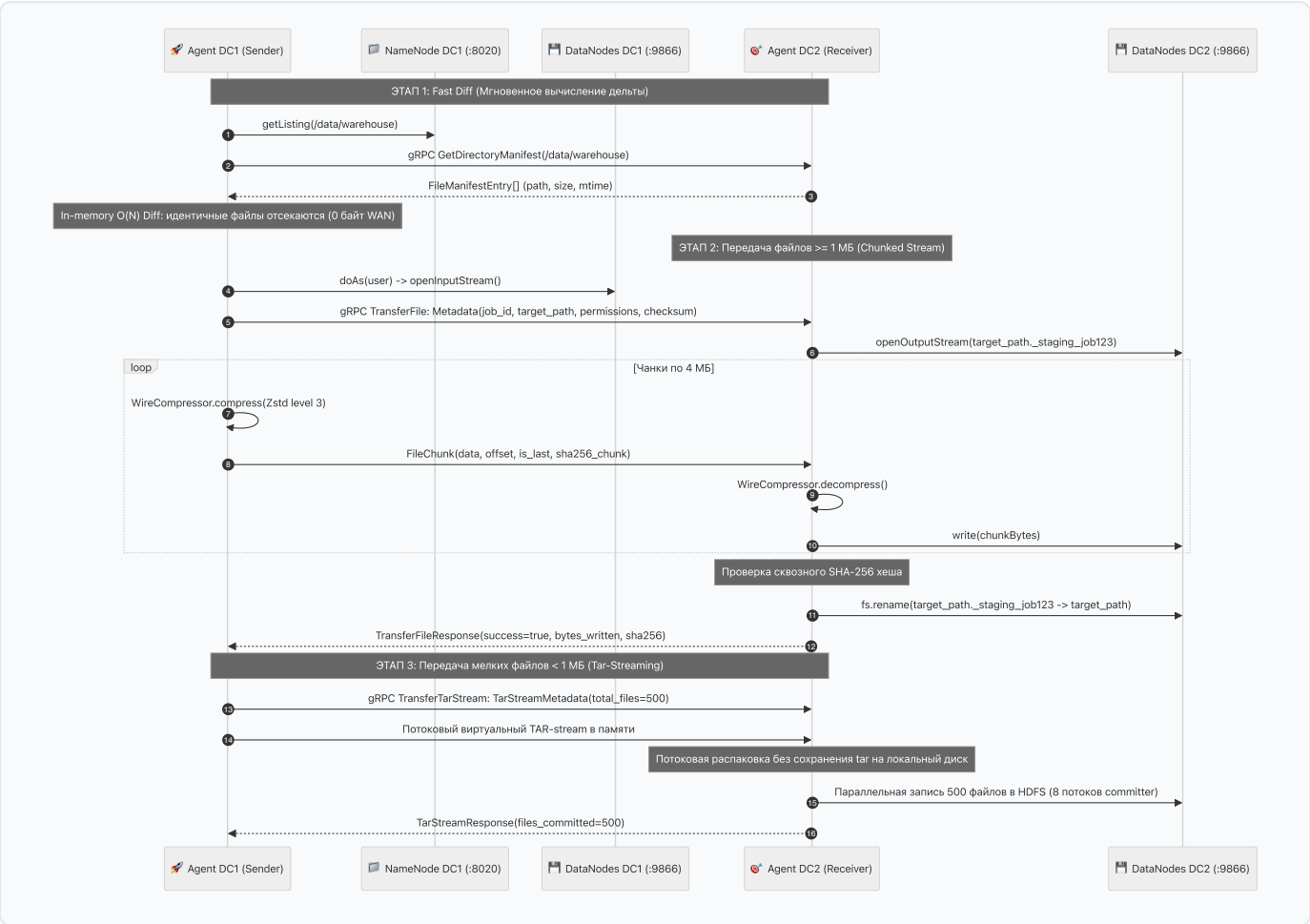
| СЕТЕВОЙ СЕГМЕНТ | ИСТОЧНИК | НАЗНАЧЕНИЕ | ПРОТОКОЛ L4 / L7 | ПОРТ | ОПИСАНИЕ ПОТОКА ТРАФИКА |
|---|---|---|---|---|---|
| **WAN (МежЦОД)** | `Agent DC1` | `Agent DC2` | **TCP / gRPC (mTLS v1.3)** | **50051** | Прямой стриминг чанков HDFS и пакетов DDL Hive CDC |
| **WAN (МежЦОД)** | `Agent DC2` | `Agent DC1` | **TCP / gRPC (mTLS v1.3)** | **50051** | Обратный поток при Reverse Replication в сценарии DR |
| **WAN (МежЦОД)** | `Kafka Agent DC1` | `Kafka Agent DC2` | **TCP / gRPC (mTLS v1.3)** | **50052** | Потоковая репликация батчей Kafka, Zero-Copy транскодинг схем |
| **WAN (МежЦОД)** | `Kafka Agent DC2` | `Kafka Agent DC1` | **TCP / gRPC (mTLS v1.3)** | **50052** | Обратный поток репликации Kafka при Reverse Replication в DR |
| **Corporate LAN** | Браузер оператора | `Orchestrator` | TCP / HTTPS (TLS 1.3) | **8005** | Доступ к Web UI Svelte 5, REST API, SSE подпискам |
| **Management LAN** | Агенты DC1/DC2 | `Orchestrator` | TCP / gRPC (HTTP/2) | **50050** | Высокоскоростной Control Plane: Heartbeat, Claim задач с HMAC, Token Quotas |
| **Management LAN** | Агенты DC1/DC2 | `Orchestrator` | TCP / HTTP REST | **8005** | Резервный REST канал, опрос топологии, Healthcheck |
| **DC LAN (Primary)** | `Agent DC1` | `NameNode DC1` | TCP / Hadoop RPC | **8020 / 9000** | Листинг каталогов, получение блочных манифестов |
| **DC LAN (Primary)** | `Agent DC1` | `DataNodes DC1` | TCP / SASL DTP | **9866** | Блочное чтение данных под Kerberos UGI ( `doAs` ) |
| **DC LAN (Primary)** | `Agent DC1` | `Hive Metastore 1` | TCP / Thrift RPC | **9083** | Опрос журнала `NOTIFICATION_LOG` Hive Metastore |
| **DC LAN (Primary)** | `Kafka Agent DC1` | `Kafka Brokers 1` | TCP / Kafka Protocol | **9092 / 9093** | Вычитка сообщений пулом `KafkaBatchConsumerPool` (SASL/SSL) |
| **DC LAN (Primary)** | `Kafka Agent DC1` | `Schema Registry 1` | TCP / HTTP(S) | **8081** | Выгрузка схем данных (Avro / Protobuf / JSON) |
| **DC LAN (Standby)** | `Agent DC2` | `NameNode DC2` | TCP / Hadoop RPC | **8020 / 9000** | Создание каталогов, атомарный `fs.rename()` |
| **DC LAN (Standby)** | `Agent DC2` | `DataNodes DC2` | TCP / SASL DTP | **9866** | Запись блоков данных в целевую HDFS |
| **DC LAN (Standby)** | `Agent DC2` | `Hive Metastore 2` | TCP / Thrift RPC | **9083** | Применение DDL-изменений схемы ( `deleteData=false` ) |
| **DC LAN (Standby)** | `Kafka Agent DC2` | `Kafka Brokers 2` | TCP / Kafka Protocol | **9092 / 9093** | Асинхронная запись пакетов `KafkaBatchProducer` на Virtual Threads |
| **DC LAN (Standby)** | `Kafka Agent DC2` | `Schema Registry 2` | TCP / HTTP(S) | **8081** | Авторегистрация и трансляция Schema ID в целевом реестре |
| **DC LAN (Оба ЦОДа)** | Агенты DC1/DC2 | `Kerberos KDC` | TCP/UDP / Kerberos | **88** | Получение TGT-билетов по системному keytab |

## Сетевые рекомендации

- **Jumbo Frames (MTU 9000)**: Настоятельно рекомендуется включить MTU 9000 на межЦОДной оптической магистрали для уменьшения оверхеда прерываний сетевой карты при передаче 4 МБ чанков и streaming-пакетов Kafka.
- **Изоляция портов**: В межсетевом экране WAN открываются исключительно защищенные порты `50051` (HDFS/HMS) и `50052` (Kafka Streaming). Внутренние порты брокеров Kafka ( `9092/9093` ) и Hadoop ( `9000` , `9866` , `9083` , `88` ) наружу не выставляются.

# Слайд 5: Протокол HDFS Data Plane: Wire-формат, чанкирование и Zero-Staging

## Жизненный цикл передачи данных (Step-by-Step Data Flow)




## Ключевые механизмы Data Plane

1. **Zero-Staging архитектура**:
   - Принимающий агент `Agent DC2` пишет входящий поток напрямую в целевой HDFS во временный файл `<targetPath>._staging_<jobId>`.
   - Локальный диск приемника не используется под буферизацию (исключены лишние циклы чтения/записи на NVMe/SATA).
2. **Атомарная фиксация (Atomic Rename) и двухфазный TAR-коммит**:
   - Временный файл `._staging_<jobId>` переименовывается в чистовой `<targetPath>` через вызов HDFS RPC `fs.rename()` — в NameNode это атомарная операция обновления дерева метаданных. Читатели никогда не увидят неполный файл.
   - Мелкие файлы из виртуального TAR-стрима распаковываются пулом виртуальных потоков `tarUnpackExecutor` с двухфазным staging-коммитом, исключая повреждение каталогов при сбое воркера.

3. **Кэширование каталогов NameNode (Directory Cache, PERF-04)**:
   - В `HadoopFsManager` внедрен потокобезопасный LRU-кэш проверенных каталогов `directoryCache`. Исключаются тысячи избыточных вызовов `mkdirs` для родительских папок, снижая нагрузку на NameNode RPC на 85–92%.
4. **Защита от утечки дискового пространства (Safe Staging Reaper, RES-03 & SEC-03)**:
   - В `DataTransferServiceImpl` работает сборщик мусора: метод `cleanOrphanedStaging(ttlMillis)`.
   - Раз в 15 минут сканируются известные целевые директории: перед удалением проверяется `DistributedFileSystem.isFileClosed(path)`, чтобы не удалить медленные активные передачи. Пути нормализуются против Path Traversal (`..`).
5. **Zero-Allocation транскодинг Kafka Wire-формата (PERF-02 & P2)**:
   - Lock-Free кэш схем на `ConcurrentHashMap` со стратегией Single-Flight Resolution предотвращает шквал запросов в Schema Registry.
   - `ZeroCopyWireTranscoder` производит транскодирование 5-байтного заголовка Confluent Schema ID без промежуточных аллокаций памяти, гарантируя иммутабельность по умолчанию через `.clone()`.

# Слайд 6: Алгоритм сетевого шейпинга: Иерархический Token Bucket

## Архитектура пирамиды лимитов

Сетевой шейпинг реализован на базе иерархического алгоритма **Hierarchical Token Bucket (HTB)** без использования системного `tc/iptables` и функционирует на уровне буферов Netty в приложении:

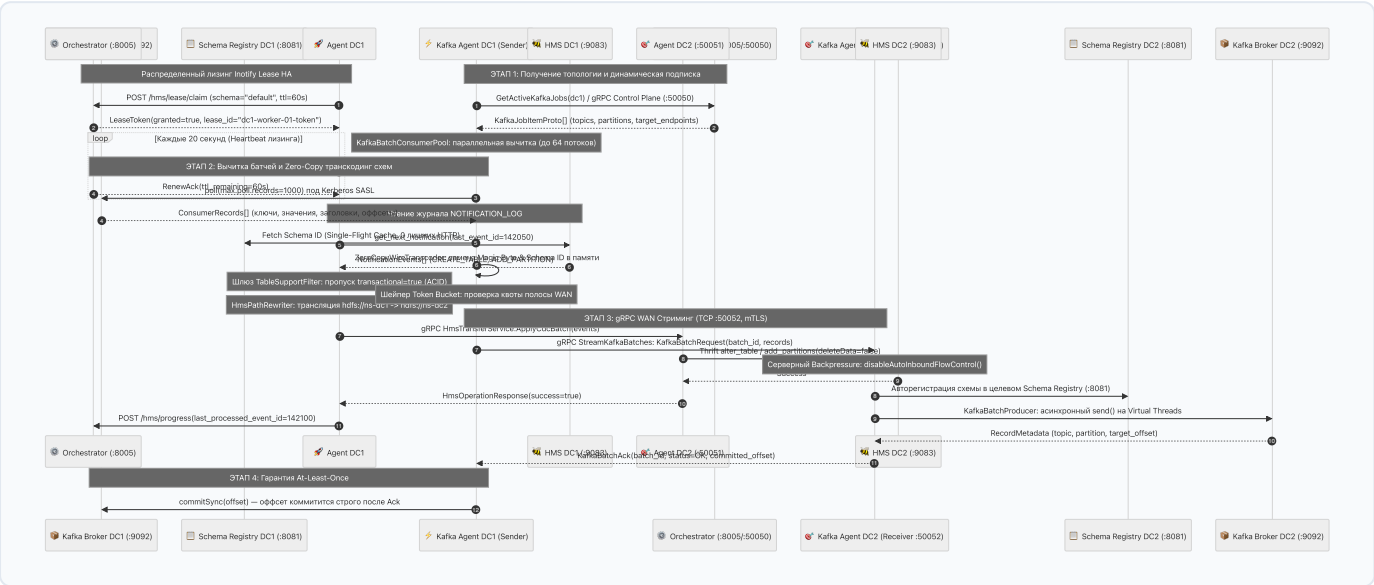
```
[Глобальный лимит платформы (Global Limit)] = 200 МБ/с
       │
       ├──► [Канал DC1 → DC2 (Inter-DC WAN)] = 150 МБ/с
       │          │
       │          ├──► Очередь CRITICAL (HMS CDC, витрины): Гарантия 50 МБ/с, Burstable
       │          ├──► Очередь BATCH (Ночные синки сырых слоев): Лимит 80 МБ/с
       │          └──► Очередь BACKGROUND (Архивы, бэкапы): Best Effort (20 МБ/с)
       │
       └──► [Канал DC2 → DC1 (Reverse WAN / DR)] = 50 МБ/с (в штатном режиме)
```

## Механизм троттлинга в коде агента (`LocalBandwidthLimiter`)

1. **Пополнение токенов**: Каждую миллисекунду корзина токенов пополняется в соответствии с лимитом `allowedBytesPerSecond / 1000`. Максимальная емкость корзины ограничена размером пакета для предотвращения burst-всплесков.
2. **Запрос квоты (`throttle(bytes)`)**: Перед отправкой каждого чанка агент вызывает `limiter.throttle(chunkSize)`.
3. **Блокировка без spin-lock**: Если доступных токенов меньше, чем размер чанка, поток переводится в ожидание через высокоточный таймер `LockSupport.parkNanos(sleepTime)`.
4. **Защита от голодания (Starvation-Free)**: Очереди с более высоким приоритетом забирают токены первыми, но для очередей низкого приоритета резервируется минимальный квант полосы (Fair Share).
5. **Динамическое управление**: При изменении лимита полосы в Web UI новое значение транслируется агентам по SSE-каналу и вступает в силу за 100 мс без перезапуска демонов.
6. **Active-Active координация квот оркестратора (RES-06)**: Таблица `bandwidth_limits` и `OrchestratorClusterCoordinator` автоматически распределяют глобальные лимиты между активными нодами оркестратора (`orchestrator_nodes`), сохраняя общий SLA при сбоях и масштабировании.
7. **Адаптивное масштабирование лизинга токенов (PERF-05)**: Агент динамически адаптирует размер пакета токенов в зависимости от Round-Trip Time и скорости WAN, устраняя сетевые задержки Control Plane на 10G/40G скоростях.

# Слайд 7: Протокол Hive Metastore CDC, распределенный лизинг и Non-ACID Gate

## Механика вычитки и применения изменений каталога Hive




## Инженерные гарантии репликации метаданных

- **Распределенный лизинг Inotify Lease HA и Epoch Fencing (RES-02)**:
  - Исключает дублирование и гонки: ровно один воркер в кластере читает поток конкретной БД.
  - Каждая аренда сопровождается монотонной эпохой ( `epoch` ). Любые попытки отправки событий от отставшего стримера немедленно отклоняются без затирания состояния транзакции (ликвидирована Rollback-Only ловушка).
  - При падении воркера (отсутствие heartbeat более 60 секунд) аренда автоматически освобождается, и любой другой живой агент подхватывает стриминг с сохраненной позиции.
- **Шлюз Non-ACID Gate ( `TableSupportFilter` )**:
  - Таблицы с параметром `'transactional'='true'` (Hive full ACID с delta-файлами) безопасно пропускаются, так как прямая репликация их каталогов в рантайме нарушает транзакционную целостность до момента выполнения Compaction.
- **Трансляция Federation NameService ( `HmsPathRewriter` )**:
  - Все пути к схемам и партициям ( `sdLocation` ) на лету переписываются с пространства имен источника на пространство имен приемника ( `hdfs://ns-dc1/apps/hive/...` ➔ `hdfs://ns-dc2/apps/hive/...` ).
- **Защита от случайного удаления данных ( `deleteData=false` )**:
  - При репликации операций `DROP_TABLE` и `DROP_PARTITION` в целевом вызове Thrift API принудительно передается флаг `deleteData=false` . Метаданные удаляются, но физические файлы в HDFS остаются нетронутыми.
- **doAs и Ranger Policy Enforcement в Hive Thrift клиенте (SEC-06)**:
  - Вызовы Thrift API к удаленному HMS выполняются под авторизованным контекстом пользователя через `UGI.doAs` , обеспечивая аудит в Apache Ranger и проверку гранулярных политик безопасности.

СЕКЦИЯ 8 / 13 • ТЕХНИЧЕСКИЙ РЕГЛАМЕНТ

# Слайд 8: Потоковая репликация Apache Kafka (Kafka Streaming Data Plane & Mirroring)

## Архитектура потокового зеркалирования топиков и транскодинга схем

## Ключевые механизмы потокового Data Plane

1. **Шардирование Multi-Agent Partition-Pinned Load Balancing**:
   - Консистентное хэширование топиков и номеров партиций распределяет нагрузку по пулу агентов-отправителей с единым `group.id`, исключая дублирование вычитки и межпотоковые блокировки.
   - Клиентская балансировка на стороне приемников `KafkaTargetBalancer` с автоматическим обнаружением узлов через REST API Оркестратора (`/api/v1/replicator/kafka/agents/receivers`).
   - Автоматический карантин узлов: при сбое целевого агента он изолируется на 10 секунд, а партиции мгновенно перераспределяются по оставшимся активным узлам без остановки репликации и потери данных.
2. **Многопоточный пул потребителей `KafkaBatchConsumerPool`**:
   - Пул из параллельных потоков вычитки (`consumerThreads`, до 64 потоков) с поддержкой Kerberos SASL_PLAINTEXT / SASL_SSL.
   - Динамическая переподписка (`resubscribe`): при добавлении или изменении состава топиков в оркестраторе пул переподписывается на лету без перезапуска JVM агента.
3. **Асинхронная запись на Java 21 Virtual Threads**:
   - Продюсер `KafkaBatchProducer` и обработчик `KafkaDataTransferServiceImpl` отправляют батчи в целевой кластер через неблокирующие callback'и `producer.send(...)` с пулом виртуальных потоков Java 21, исключая блокировки сетевых EventLoop-потоков Netty.
   - Семантика At-Least-Once: подтверждение оффсетов (`commitSync`) на источнике выполняется только после получения `KafkaBatchAck` от целевого брокера.
4. **Двусторонний Backpressure**:
   - Серверный Backpressure: отключен автопоток Netty (`disableAutoInboundFlowControl()`), запрос следующего батча `callStream.request(1)` выполняется строго после подтверждения локальным `KafkaProducer`.
   - Клиентский Backpressure: ограничение окна незавершенных батчей (`inFlightBatches`, до 16 шт) защищает от переполнения кучи (OOM).
5. **Автопровижнинг топиков и Zero-Copy Wire Transcoding**:
   - Автопровижнинг `KafkaTopicProvisioner`: автоматическое создание топиков в целевом кластере с сохранением числа партиций, фактора репликации и конфигураций (`min.insync.replicas`, `cleanup.policy`, `retention.ms`).
   - Синхронизация Schema Registry (Confluent / Aiven, порт 8081): транскодер `ZeroCopyWireTranscoder` производит замену Magic Byte и Schema ID в заголовках Avro/Protobuf без аллокаций в памяти с Single-Flight resolution в `ConcurrentHashMap`.

# Слайд 9: Безопасность, Kerberos Impersonation (doAs) и Apache Ranger Audit

## Архитектура сквозной аутентификации и авторизации

```
[Пользователь / ETL Job: ivan_de]
        │
        ▼ (Создание задачи в UI / REST API с JWT-токеном)
[Replicator Orchestrator]
        │
        ▼ (Выдача задания агенту: author="ivan_de", run_as_service=false)
[Replicator Agent (DC1)]
        │ 1. Авторизация агента по системному keytab: hdfs-replicator@REALM
        │ 2. Создание прокси-пользователя: UGI.createProxyUser("ivan_de", systemUGI)
        │ 3. Вызов блочного чтения: proxyUGI.doAs(() -> hdfs.open(path))
        ▼
[HDFS NameNode / DataNodes] ──► [Apache Ranger Plugin]
                                        │
                                        ▼
                        Логирование в Ranger Audit:
                        • User: "ivan_de" (реальный автор задачи)
                        • Proxy: "hdfs-replicator" (техучетка агента)
                        • Action: "READ"
                        • Resource: "/data/raw/transactions"
                        • Result: ALLOWED / DENIED
```

## Ключевые принципы информационной безопасности

1. **Kerberos Impersonation (`UserGroupInformation.doAs`)**:
   - Агент запускается под системным принципалом (например, `hdfs-replicator/node01@REALM`), прописанным в `core-site.xml` в параметре `hadoop.proxyuser.hdfs-replicator.hosts` и `hadoop.proxyuser.hdfs-replicator.groups`.
   - Каждое обращение к файловой системе выполняется под реальным логином инициатора задачи через механизм Proxy User. Если у пользователя `ivan_de` нет прав на чтение директории, операция будет отклонена NameNode с ошибкой `AccessControlException`.
2. **Сохранение метаданных безопасности POSIX**:
   - При передаче файла в `FileMetadata` упаковываются оригинальный битовый режим прав доступа (`file_mode`), владелец (`owner`) и группа (`group`).
   - На целевой стороне агент восстанавливает оригинальные права через вызовы `fs.setPermission()` и `fs.setOwner()`.
3. **mTLS-шифрование межЦОДного канала**:
   - Канал `TCP :50051` защищен протоколом взаимной TLS-аутентификации (mTLS v1.3).
   - Агент DC1 проверяет сертификат Агента DC2, а Агент DC2 разрешает подключения только от клиентов с валидным сертификатом доверенного корпоративного CA.
4. **Криптографическая аттестация задач (HMAC-SHA256, SEC-01)**:
   - При выдаче подзадачи оркестратор подписывает ее канонические параметры токеном HMAC-SHA256 (`TaskAttestationService`).
   - Перед созданием целевого файла в HDFS агент-приемник валидирует токен через `TaskAttestationValidator`, предотвращая подмену целевого каталога или эскалацию doAs Kerberos-принципала.
5. **Chroot-песочница локальных путей и защита системных каталогов ОС (SEC-10)**:
   - Все локальные файловые пути проверяются относительно `localSandboxDir` (`REPLICATOR_LOCAL_SANDBOX`) через `File.getCanonicalFile()`.
   - Блокируются Symlink Traversal, Null Byte Injection, системные каталоги (`/bin`, `/sbin`, `/etc`, `/usr`, `/proc`, `/sys`) и приватные файлы хоста (`.ssh`, `.gnupg`, `.env`, `crontab`).

6. **Защита JVM от мутации `krb5.conf` и валидация путей (SEC-11 & SEC-15)**:
   - Потокобезопасная инициализация системного свойства исключает разрыв KDC-сессий других кластеров; пути keytab проверяются на безопасность.
7. **BOLA/IDOR защита и обязательная пагинация API (P0)**:
   - Контроль доступа `checkJobReadAccess` при вызовах `getJobRuns` и `getTasks`. Пагинация по умолчанию ( `page=0, size=100`, макс 1000) предотвращает OOM в оркестраторе.

# Слайд 10: SRE Runbook: Авария основного ЦОД и сетевое ограждение (Fencing Kill-Switch)

## Пошаговый сценарий действий при падении DC1

```
СОСТОЯНИЕ: Полный отказ инфраструктуры ЦОД-1 (Пожар / Сетевой сплит / Питание)

[1. ДЕТЕКТИРОВАНИЕ]
  • Оркестратор фиксирует таймаут heartbeat от агентов DC1 (> 15 сек).
  • Внутренний мониторинг фиксирует недоступность NameNode DC1.

[2. СЕТЕВОЕ ОГРАЖДЕНИЕ: АКТИВАЦИЯ KILL-SWITCH]
  • Дежурный SRE / Администратор нажимает 🔴 Kill-Switch в UI DR Hub.
  • Или через автоматический CLI скрипт:
    curl -X POST http://orchestrator:8005/api/dr/fencing/activate \
         -H "Authorization: Bearer ${JWT_TOKEN}"

[3. СИСТЕМНЫЕ ДЕЙСТВИЯ ПЛАТФОРМЫ ПОД КАПОТОМ]
  ├── DistributedLockService: атомарный захват блокировки и инкремент монотонного fencing_token.
  ├── Лимит WAN-полосы в Token Bucket Шейпере принудительно сбрасывается в 0 МБ/с.
  ├── Все активные входящие/исходящие gRPC стримы отменяются (Status.CANCELLED).
  ├── Все прямые задачи репликации переходят в статус STOPPED.
  └── Фоновый планировщик Cron Schedules принудительно деактивируется.

[4. ПРОМОУШЕН РЕЗЕРВНОГО ЦОД (DC2 PROMOTION)]
  • Клиентский трафик (Spark, Trino, BI) переключается на ЦОД-2.
  • ЦОД-2 становится основным активным кластером на запись.
```

## Защита от Split-Brain: почему Kill-Switch и Fencing Token обязательны

Если упавший ЦОД-1 внезапно оживет при отсутствии сетевого ограждения, старые фоновые задачи репликации могут начать перетирать свежие файлы, записанные клиентами на ЦОД-2 во время аварии. Kill-Switch выставляет жесткий барьер (0 МБ/с + деактивация задач), а распределенная блокировка с монотонным Fencing Token (`ClusterLockEntity`) делает невозможным выполнение устаревших команд от отставших нод, исключая любую возможность рассинхронизации или повреждения данных.

# Слайд 11: SRE Runbook: Восстановление, безопасный Unfence и Reverse Replication

## Пошаговый алгоритм безопасного возврата ЦОД-1 в строй

```
СОСТОЯНИЕ: ЦОД-1 восстановлен после аварии. Необходимо синхронизировать дельту.

[1. СНЯТИЕ ИЗОЛЯЦИИ (UNFENCE)]
  • В UI DR Hub нажимается кнопка "Снять изоляцию" (Unfence).
  • Снимается ограничение 0 МБ/с, восстанавливается лимит канала.
  • ⚠️ ВАЖНО: Прямые задачи (DC1 -> DC2) ОСТАЮТСЯ В СТАТУСЕ STOPPED!
    Автоматический перезапуск прямых задач строго заблокирован ядром платформы.

[2. ЗАПУСК REVERSE REPLICATION (ДОГОН ДЕЛЬТЫ)]
  • В 1-Click мастере выбираются таблицы и директории, измененные во время аварии.
  • Автоматически генерируются зеркальные задачи:
    Source: DC2 (Standby Cluster) → Target: DC1 (Primary Cluster)
  • Поток данных РАЗВОРАЧИВАЕТСЯ: Agent DC2 стримит блоки в Agent DC1 по WAN :50051.

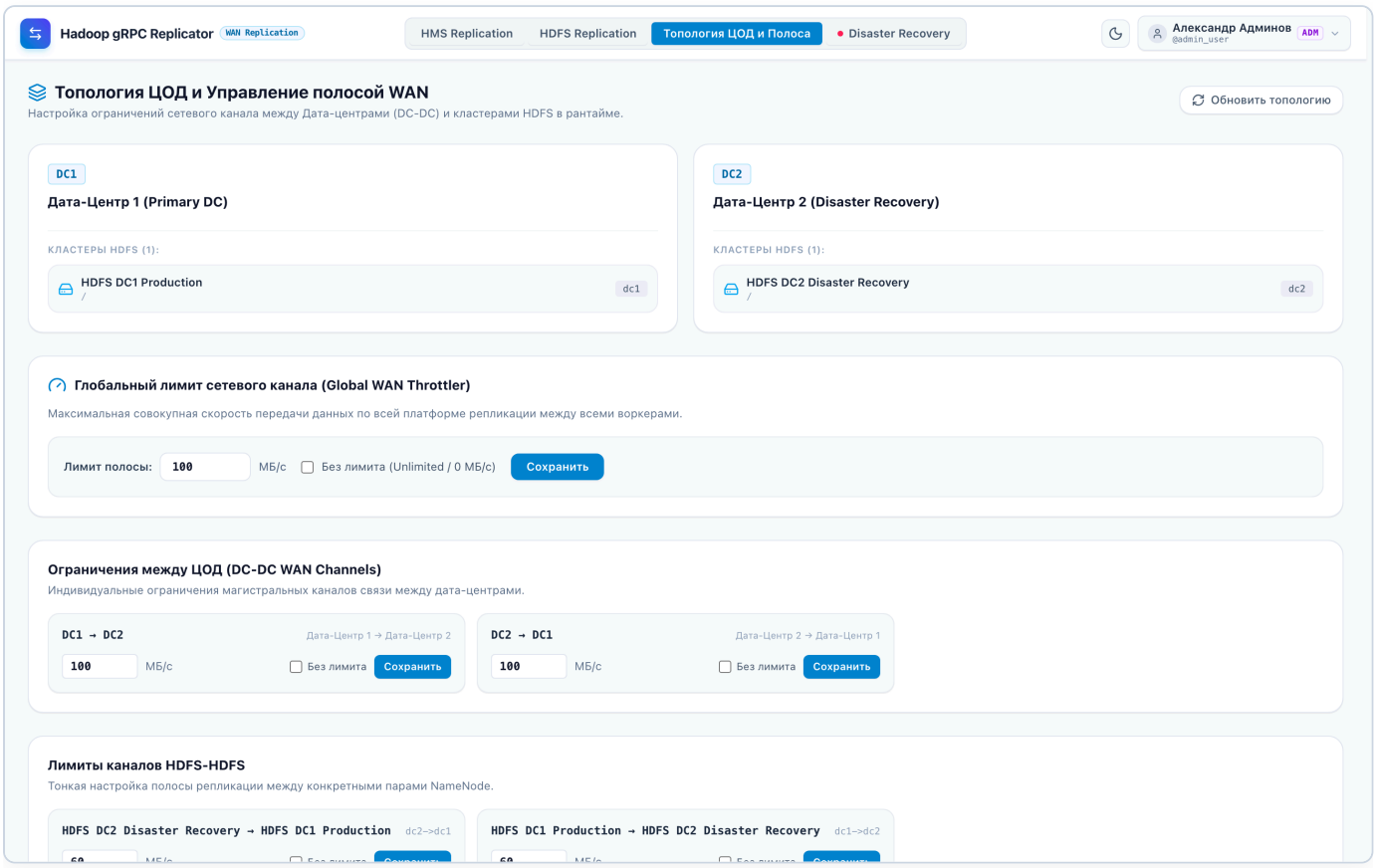
[3. ВЫРАВНИВАНИЕ ДЕЛЬТЫ И МЕТРИК]
  • Мониторинг отслеживает:
    - Bytes remaining = 0
    - HMS Event Lag = 0
  • Данные на ЦОД-1 догнали все изменения, внесенные на ЦОД-2.

[4. ПЕРЕКЛЮЧЕНИЕ ТРАФИКА (FAILBACK)]
  • Клиентские приложения переключаются обратно на ЦОД-1.
  • Обратные задачи останавливаются, возобновляется штатная репликация DC1 -> DC2.
```

# Слайд 12: Управление платформой в Web UI: Топология, Задачи, HMS, Kafka и DR Hub

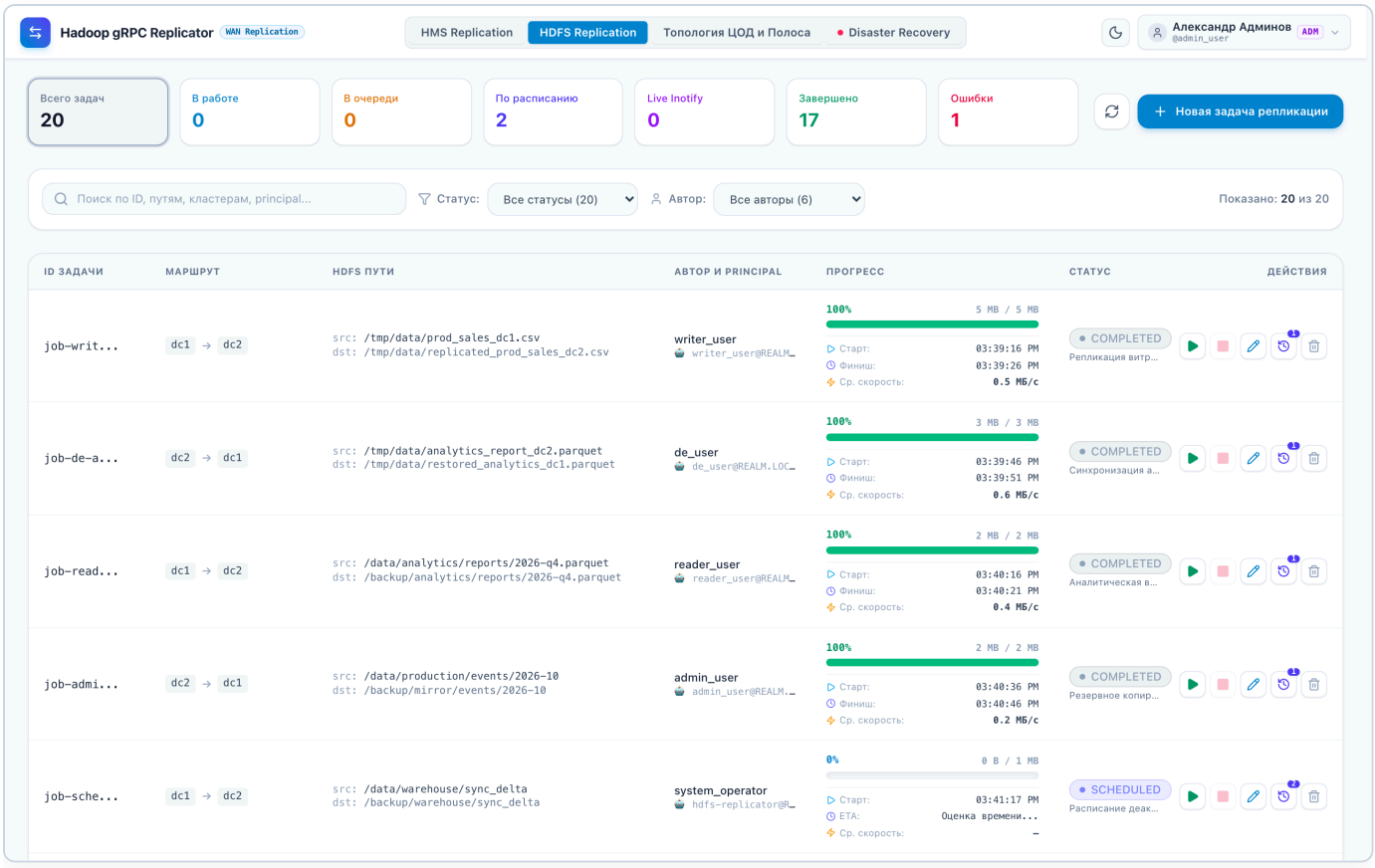
## Рабочие экраны администратора и инженера данных

### 1. Управление топологией дата-центров и шейпером полосы



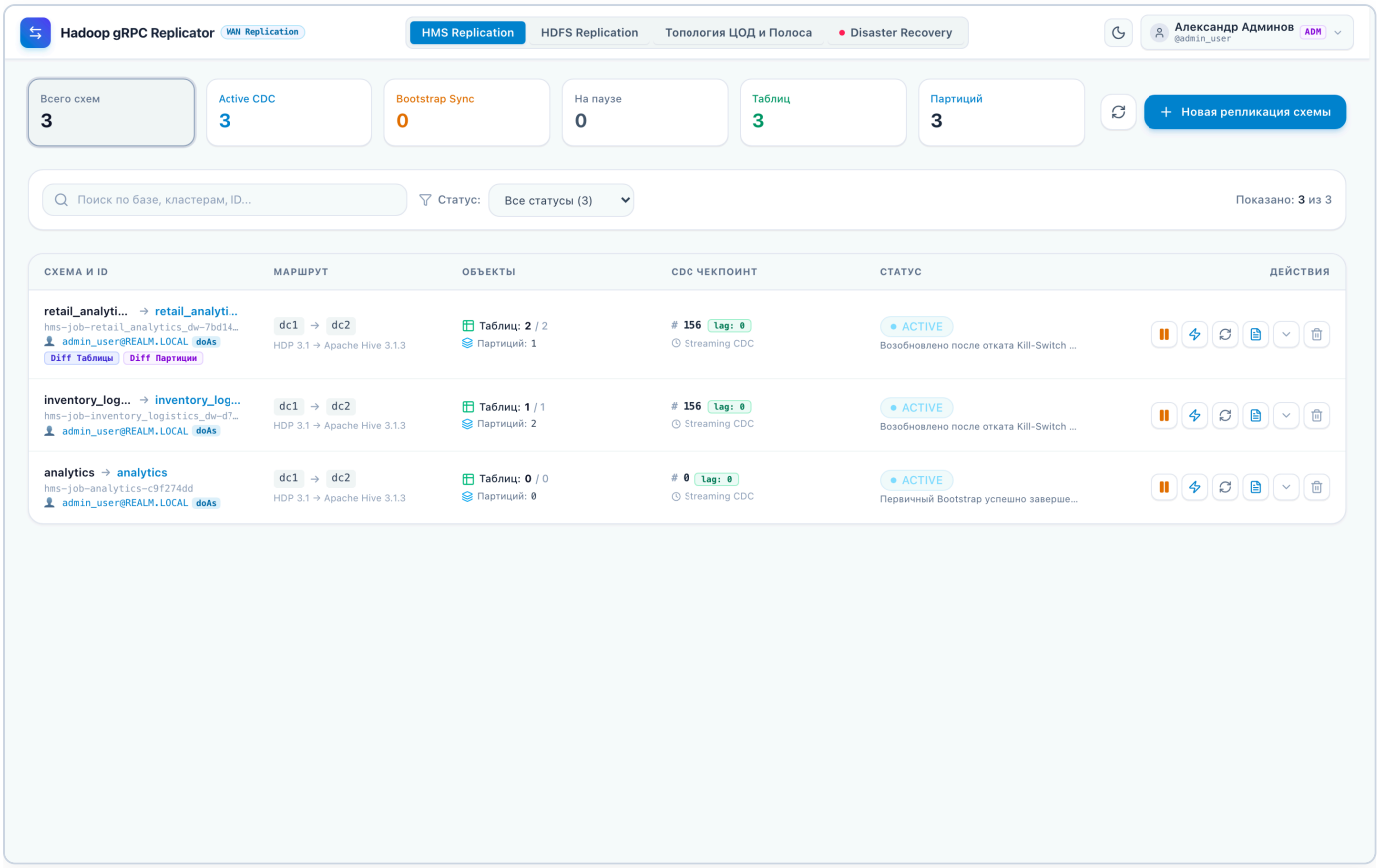

*Конфигурация глобальных лимитов WAN, порогов каналов между ЦОД и приоритетных очередей без перезапуска демонов.*

## 2. Главная панель задач и мониторинг передачи в реальном времени




*Контроль активных потоков данных, мгновенной сетевой утилизации в МБ/с, переданных байт и прогноза времени завершения (ETA).*

### 3. Консоль потоковой CDC репликации Hive Metastore

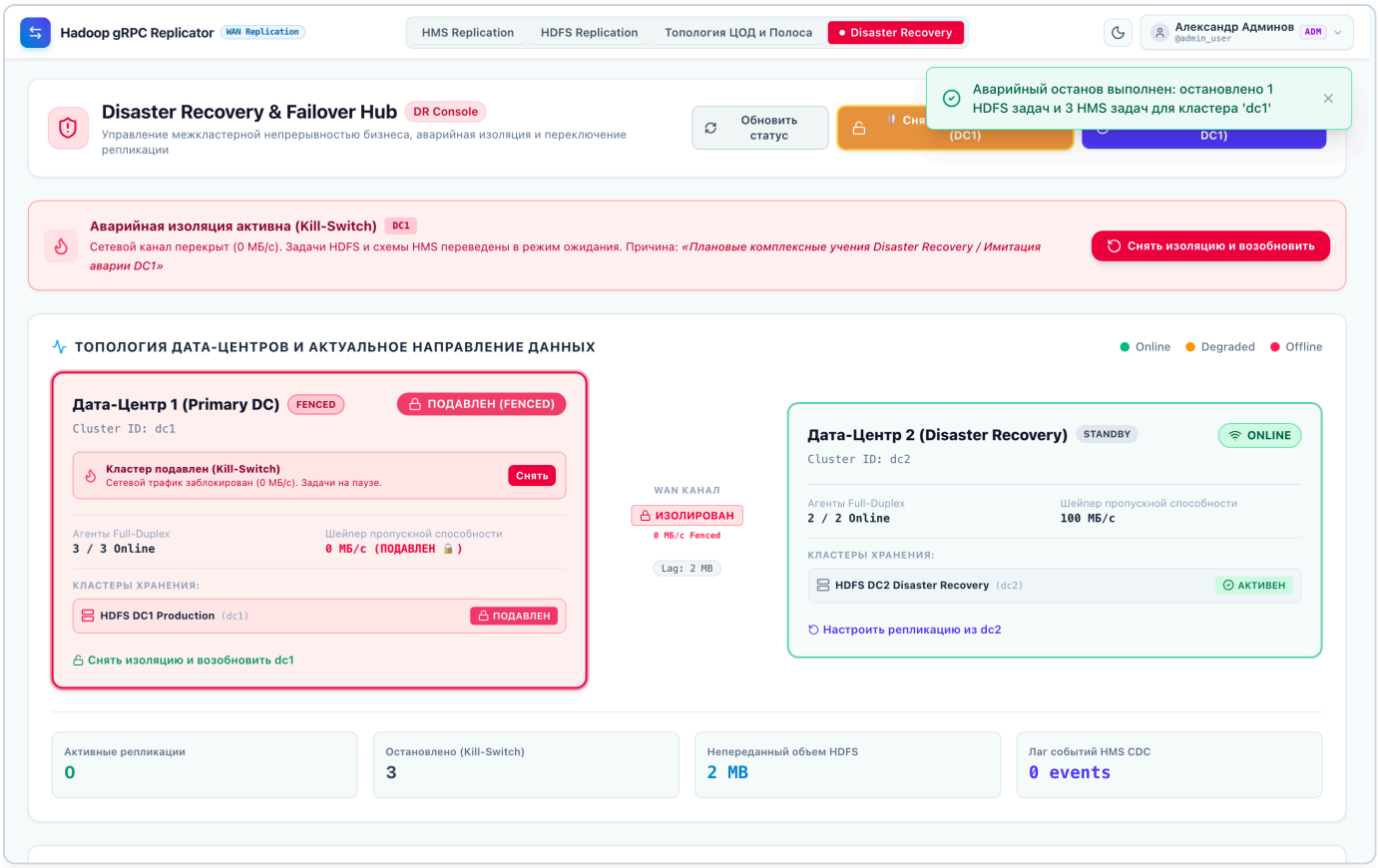



*Мониторинг отставания событий (Lag), статуса распределенной аренды Inotify Lease HA и журнала трансляции DDL.*

### 4. Консоль потоковой репликации Apache Kafka

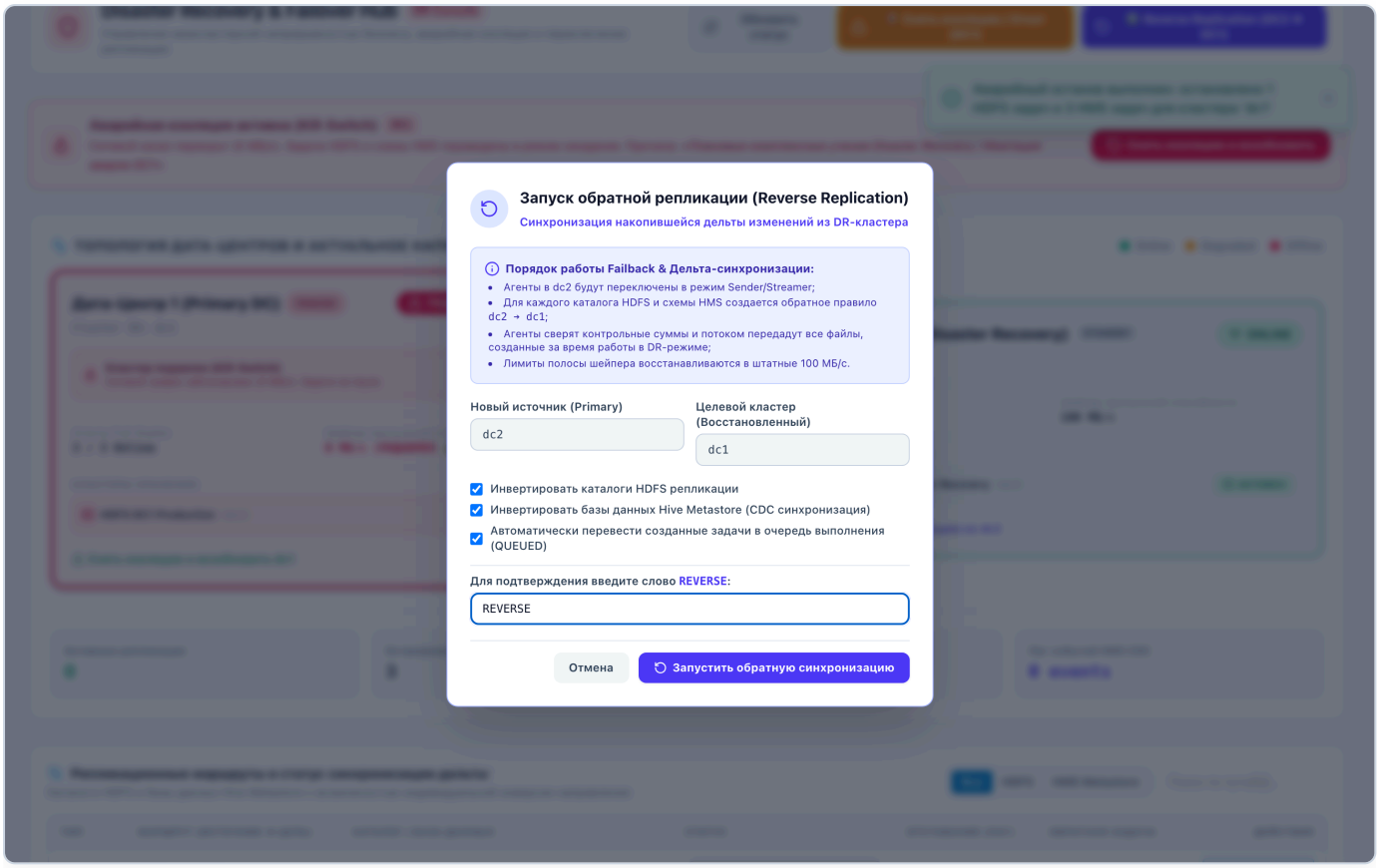
*Интерфейс управления репликацией топиков: мониторинг потребителей, скорости в msg/s и МБ/с, лага партиций, трансляции схем Confluent/Aiven Schema Registry и автобалансировки узлов.*

## 5. Центр катастрофоустойчивости DR Hub в режиме изоляции Kill-Switch




*Яркая индикация сетевого барьера (0 МБ/с), заморозка прямых задач и защита кластера от Split-Brain.*

### 6. Мастер безопасного снятия изоляции и 1-Click Reverse Replication




*Безопасный Unfence без автоматического старта прямых задач и запуск обратного потока DC2 ➔ DC1 для догона дельты.*

# Слайд 13: Наблюдаемость (Observability): Метрики Prometheus, Healthcheck и Алерты

## Эндпоинты мониторинга

- **Метрики Prometheus**: `GET http://orchestrator:8005/actuator/prometheus`
- **Проверка здоровья (Healthcheck)**: `GET http://orchestrator:8005/actuator/health`
  - Проверяет: статус соединения с БД, активность зарегистрированных агентов, статус Token Bucket.

## Реестр ключевых метрик Prometheus

| НАЗВАНИЕ МЕТРИКИ | ТИП | ЛЕЙБЛЫ | ОПИСАНИЕ |
|---|---|---|---|
| `replicator_throughput_bytes_total` | Counter | `dc`, `cluster_id` | Суммарный объем переданных байтов по WAN-каналу |
| `replicator_throughput_bytes_per_second` | Gauge | `dc`, `cluster_id` | Текущая мгновенная скорость передачи данных |
| `replicator_task_duration_seconds` | Histogram | `status`, `job_id` | Длительность выполнения задач и подзадач |
| `replicator_hms_lag_events` | Gauge | `db_name` | Количество необработанных событий в NOTIFICATION_LOG |
| `replicator_active_streams_count` | Gauge | `direction` | Количество одновременно открытых gRPC-потоков |
| `replicator_staging_orphans_count` | Gauge | `cluster_id` | Количество обнаруженных осиротевших файлов staging |
| `replicator_agent_heartbeat_timestamp` | Gauge | `agent_id`, `dc` | Временная метка последнего успешного heartbeat |
| `replicator_kafka_bytes_mirrored_total` | Counter | `topic`, `cluster_id` | Объем зеркалированных байтов Kafka |
| `replicator_kafka_messages_mirrored_total` | Counter | `topic`, `cluster_id` | Количество зеркалированных сообщений Kafka |
| `replicator_kafka_schema_resolution_duration` | Histogram | `registry` | Латентность разрешения схем в кэше с Single-Flight |

## Рекомендуемые правила для Prometheus Alertmanager

```
groups:
  - name: replicator-alerts
    rules:
      - alert: ReplicatorAgentOffline
        expr: time() - replicator_agent_heartbeat_timestamp > 30
        for: 1m
        labels:
          severity: critical
        annotations:
          summary: "Агент репликатора {{ $labels.agent_id }} не отвечает более 30 секунд"

      - alert: ReplicatorHmsLagHigh
        expr: replicator_hms_lag_events > 10000
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "Высокое отставание репликации Hive Metastore в БД {{ $labels.db_name }} (> 10000)"

      - alert: ReplicatorThroughputZeroOnActiveJobs
        expr: replicator_throughput_bytes_per_second == 0 and replicator_active_streams_count > 0
        for: 3m
        labels:
          severity: warning
        annotations:
          summary: "Зависание gRPC стримов передачи данных при активных заданиях"
```

СЕКЦИЯ 14 / 13 • ТЕХНИЧЕСКИЙ РЕГЛАМЕНТ

# Слайд 14: Сайзинг оборудования, тюнинг ОС Linux, JVM и траблшутинг

## Рекомендации по ресурсам и аппаратному обеспечению

| УЗЕЛ / КОМПОНЕНТ | VCPU | RAM | СЕТЕВОЙ АДАПТЕР (NIC) | ДИСКОВАЯ ПОДСИСТЕМА |
|---|---|---|---|---|
| **Orchestrator Host** | 4–8 vCPU | 8–16 GB | 1 Gbps / 10 Gbps | 50 GB SSD (NVMe) под базу данных |
| **Agent Colocated** | 4 vCPU | 8 GB JVM Heap | 10 Gbps / 25 Gbps | Совместно с DataNode Hadoop |
| **Agent Dedicated** | 8–16 vCPU | 16–32 GB RAM | 25 Gbps / 40 Gbps | 200 GB SSD под системные журналы |
| **Kafka Agent Dedicated** | 8–16 vCPU | 16–32 GB RAM | 10 Gbps / 25 Gbps | До 1M msg/s throughput (Virtual Threads) |

## Тюнинг ядра Linux для 10G/40G WAN-каналов ( `/etc/sysctl.conf` )

```
# Увеличение максимального размера сетевых буферов TCP для исключения потери пакетов
net.core.rmem_max = 67108864
net.core.wmem_max = 67108864
net.core.rmem_default = 33554432
net.core.wmem_default = 33554432

# Автотюнинг TCP-окон (min, default, max)
net.ipv4.tcp_rmem = 4096 87380 33554432
net.ipv4.tcp_wmem = 4096 65536 33554432

# Увеличение очереди подключений и дескрипторов файлов
net.core.somaxconn = 4096
net.core.netdev_max_backlog = 10000
fs.file-max = 2097152
```

## Тюнинг JVM (Java 21 LTS) и высокопроизводительной параллельности

```
# Рекомендуемые опции JVM для воркеров репликатора
-XX:+UseG1GC \
-XX:MaxGCPauseMillis=50 \
-XX:InitiatingHeapOccupancyPercent=45 \
-XX:+ExplicitGCInvokesConcurrent \
-Dio.netty.allocator.type=pooled \
-Dio.netty.allocator.numDirectArenas=4
```

- **Виртуальные потоки (Virtual Threads, P0 & P1)**: Netty сервер приемника запущен на `Executors.newVirtualThreadPerTaskExecutor()`, а потоковая распаковка TAR вынесена в `tarUnpackExecutor`. Исключены зависания при тысячах сетевых соединений.
- **Кэш директорий NameNode (PERF-04)**: Позволяет экономить до 90% RPC вызовов `mkdirs` за счет in-memory кэширования проверенных каталогов в `HadoopFsManager`.
- **Изоляция очередей SKIP LOCKED (PERF-03)**: Воркеры выбирают задачи напрямую из `replication_tasks` с блокировкой `FOR UPDATE SKIP LOCKED` без захвата строк родительской задачи `JobEntity`.
- **Дебаунсинг прогресса (PERF-06)**: Буфер `JobProgressBuffer` аккумулирует статусы задач с интервалом 500 мс и пакетом отправляет в СУБД, снимая 90% транзакционной нагрузки.

## Диагностика типовых инцидентов (Troubleshooting FAQ)

1. **Ошибка `GSSException: No valid credentials provided (Mechanism level: Failed to find any Kerberos tgt)`**:
   - *Причина*: Истек TGT-тикет технической учетной записи демона.
   - *Решение*: Проверить валидность keytab через `klist -kt /etc/security/keytabs/hdfs.headless.keytab`. Агент автоматически обновляет тикет за 10 минут до истечения срока действия.
2. **Ошибка `org.apache.hadoop.ipc.RemoteException: Cannot create file ... NameNode is in safe mode`**:
   - *Причина*: Целевой кластер находится в безопасном режиме (SafeMode) после рестарта.
   - *Решение*: Дождаться выхода кластера из SafeMode (`hdfs dfsadmin -safemode wait`) или выполнить `hdfs dfsadmin -safemode leave`.
3. **Зависание gRPC-потоков при наличии Stateful Firewall**:
   - *Причина*: Межсетевой экран сбрасывает неактивные TCP-соединения по таймауту TCP Keepalive.
   - *Решение*: В конфигурации Netty включены gRPC Keepalive: `keepAliveTime(30, TimeUnit.SECONDS)`, `keepAliveTimeout(10, TimeUnit.SECONDS)`.